



Overfitting and Regularization

Why a Perfect Fit Is a Warning Sign, and What to Do About It

Rokan Uddin Faruqui

Professor, Department of CSE

University of Chittagong

rufaruqui@cu.ac.bd

Week 3 · Part 2

Where we stand

We can now fit

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b \quad (\text{a number})$$

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b) \quad (\text{a class})$$

and we can manufacture as many features as we like — x^2 , x^3 , x_1x_2 , ...

Week 2 ended with a warning: adding features always lowers the training cost. So the training cost cannot tell us when to stop. Today we look at what actually goes wrong, and at the standard remedy.

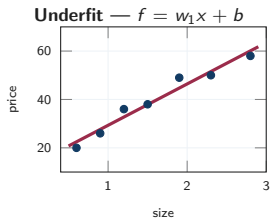
Today

1. Underfitting, overfitting, generalization
2. Overfitting in classification
3. Three ways to address it
4. The regularized cost and λ
5. Regularized linear regression
6. Regularized logistic regression

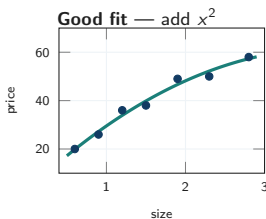
This closes Module 1: everything here applies unchanged to the neural networks of Module 2.

The Problem of Overfitting

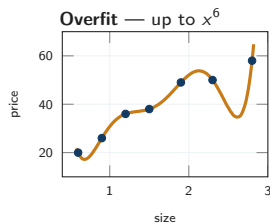
Three fits to the same seven flats



$$J_{\text{train}} = 3.19$$



$$J_{\text{train}} = 1.44$$



$$J_{\text{train}} = 0.00$$

Seven points, seven parameters: the degree-6 curve passes through every one, so the training cost is exactly zero — and it predicts that a 2.6k sq ft flat is worth *less* than a 2.1k sq ft one.

Key idea

The lowest training cost is the worst model. What we want is not a fit but **generalization**: low cost on examples the model has never seen.

	Underfitting	Overfitting
Also called	high bias	high variance
Symptom	poor on training <i>and</i> on new data	excellent on training, poor on new data
What it means	a strong preconception the data contradicts	the noise has been fitted as if it were signal
Cause	too few features, too simple a form	too many features for the amount of data

Why “variance”

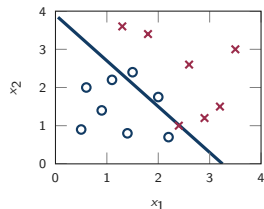
Re-collect the training data and refit. The degree-6 curve would come out completely different every time; the straight line would barely move. High variance means the fitted function depends violently on which examples you drew.

Common pitfall

You cannot detect overfitting from J_{train} — by construction it looks best when the problem is worst. You need data the model did not train on. Module 2 builds this into a train / cross-validation / test procedure.

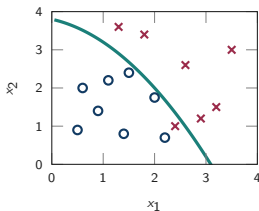
The same thing in classification

Underfit: $z = w_1x_1 + w_2x_2 + b$



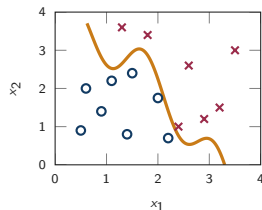
3 training points wrong

Good fit: add x_1^2, x_2^2



1 training point wrong

Overfit: high-order terms



0 training points wrong

Same fifteen patients, three boundaries. Judged on the training set the right-hand model is the best of the three — and it is the one you would least want in a clinic. It has bent itself around individual examples, so a new patient landing in one of its inlets is classified by an accident of who happened to be in the training set.

Key idea

Overfitting is not about regression or classification. It is about a model flexible enough to memorise the examples instead of describing the pattern behind them.

Addressing Overfitting

1. Collect more data

The most effective fix when it is available. With enough examples even a high-order polynomial cannot wiggle — there is a point in the way of every detour.

Limit: often impossible. There may only be 200 patients.

2. Use fewer features

Select a subset — by domain knowledge, or systematically. Fewer parameters, less room to memorise.

Limit: you throw away information. If all 100 features carry a little signal, discarding 90 discards signal with the noise.

3. Regularization

Keep every feature, but force the weights to stay small. A gentler version of feature selection: the model may use a feature, but only if it earns its place.

This is the subject of the rest of the lecture.

Key idea

Why do small weights help? A large w_j makes the prediction swing wildly for a small change in x_j — that is exactly the wiggle. In the degree-6 fit two slides back, $\sum_j w_j^2 \approx 1.2 \times 10^8$, with $|w_2| \approx 7153$. The wiggle and the huge coefficients are the same phenomenon.

The Cost Function with Regularization

Add a term to the cost that grows when the weights do:

$$J(\mathbf{w}, b) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (f_{\mathbf{w}, b}(x^{(i)}) - y^{(i)})^2}_{\text{fit the data}} + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{keep } \mathbf{w} \text{ small}}$$

- $\lambda \geq 0$ is the **regularization parameter**. You choose it; it is not learned.
- Dividing by the same $2m$ makes a good λ less sensitive to the size of the training set.
- The sum runs $j = 1, \dots, n$: **b is not penalised**.
Shrinking the intercept towards zero says the output should be near zero by default, which is a claim about the world, not about complexity. In practice it makes almost no difference; convention is to leave it out.

The trade-off

Minimising this cost balances two goals that pull in opposite directions:

- fit the training data,
- keep every w_j small.

λ sets the exchange rate between them.

This penalty is called **L2** regularization, after the ℓ_2 norm $\|\mathbf{w}\|_2^2 = \sum_j w_j^2$. In statistics the resulting model is *ridge regression*.

What λ does, at the two extremes

$$\lambda = 0$$

The penalty vanishes; we are back to the unregularized fit. The degree-6 curve returns, with $J_{\text{train}} = 0$ and coefficients in the thousands. **Overfits.**

$$\lambda \rightarrow \infty \text{ (say } 10^{10})$$

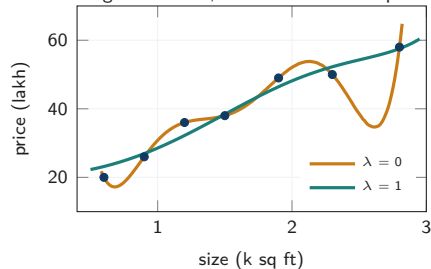
The only way to keep J small is $w_j \approx 0$ for every j , leaving $f_{\mathbf{w},b}(\mathbf{x}) \approx b$. The model predicts the same number for every input — a horizontal line. **Underfits.**

In between

Some intermediate λ balances the two. Finding it is a search, like the search for α ; Module 2 gives the principled procedure.

Both curves use *all six* powers of x . With $\lambda = 1$ the fitted weights collapse from $\sum_j w_j^2 \approx 1.2 \times 10^8$ to 33.2, the largest from 7153 to 3.37, and J_{train} rises from 0 to 2.67. The features were never removed — they were merely made expensive.

The same degree-6 model, with and without a penalty



Regularized Linear Regression

The gradient, and the update

Differentiating the regularized cost adds one term, and only to the weights:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}).$$

Regularized gradient descent

Repeat, simultaneously for $j = 1, \dots, n$:

$$w_j := w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right], \quad b := b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w},b}(x^{(i)}) - y^{(i)})$$

Common pitfall

The b update is unchanged. Adding $\frac{\lambda}{m} b$ to it is the single most common implementation error here, and it is silent — the model still trains, just towards a slightly wrong intercept.

What the extra term actually does

Collect the w_j terms in the update:

$$w_j := \underbrace{\left(1 - \alpha \frac{\lambda}{m}\right)}_{\text{shrink}} w_j - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^m \left(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\right) x_j^{(i)}}_{\text{the usual step}}$$

Every iteration now does two things: it multiplies w_j by a number slightly less than one, then takes the ordinary gradient step.

With $\alpha = 0.01$, $\lambda = 1$, $m = 50$:

$$1 - \alpha \frac{\lambda}{m} = 1 - \frac{0.01}{50} = 0.9998$$

Tiny per step — but $0.9998^{1000} = 0.8187$. Left alone for a thousand iterations, a weight decays to 82% of its value.

Key idea

This is why L2 regularization is also called **weight decay**. Each weight leaks away a little every iteration, and survives only if the data keeps pushing it back up. A feature that does not help is quietly driven towards zero.

The shrink factor must stay in $(0, 1)$: if $\alpha\lambda/m > 1$ the sign of w_j flips each iteration and training diverges.

Regularized Logistic Regression

The same penalty, the same update

Add the identical term to the logistic cost of Part 1:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log f^{(i)} + (1 - y^{(i)}) \log(1 - f^{(i)}) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2,$$

with $f^{(i)} = g(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)$. Differentiating gives

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f^{(i)} - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f^{(i)} - y^{(i)}).$$

Key idea

Character for character the regularized linear-regression gradient. The only difference, again, is the definition of f . One implementation of `gradient(X, y, w, b, lam)` serves both models.

In code

```
dj_dw = X.T @ err / m + lam * w / m  
dj_db = err.mean()           # no lam
```

Geometrically, shrinking $\mathbf{w}$ in a logistic model flattens the sigmoid along the boundary — predictions become less confident and the boundary straighter.

Choosing λ

Search on a multiplicative grid, as with α :

0, 0.01, 0.03, 0.1, 0.3, 1, 3, 10, ...

For each, train to convergence and measure the cost on data held out from training. Pick the λ with the lowest held-out cost — *never* the lowest training cost, which always chooses $\lambda = 0$.

Common pitfall

Regularization penalises w_j , whose size depends on the units of feature j . Without feature scaling the penalty falls unequally on features measured in square feet and features measured in rooms. **Scale first, always.**

L1 — a different penalty

Replace $\sum_j w_j^2$ with $\sum_j |w_j|$ and the behaviour changes qualitatively: weights are driven to *exactly* zero rather than merely made small.

- **L2 (ridge)**: shrinks everything, keeps everything.
- **L1 (lasso)**: performs feature selection as a side effect — useful when you need to know which features matter.

L2 is the default and the only one used in this course.

Key idea

Regularization does not make a bad model good. It buys back some of the flexibility you spent on extra features.

Wrap-up and Assessment

Concepts

- Underfitting is high bias; overfitting is high variance. Zero training cost is a symptom, not an achievement.
- The goal is generalization, which J_{train} cannot measure.
- Remedies: more data, fewer features, or regularization.
- L2 adds $\frac{\lambda}{2m} \sum_j w_j^2$; b is left out.
- The update becomes shrink-then-step — weight decay.
- Linear and logistic regression share one regularized gradient.

Formulas

$$J = \frac{1}{2m} \sum_i \left(f^{(i)} - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$$w_j := \left(1 - \alpha \frac{\lambda}{m} \right) w_j - \frac{\alpha}{m} \sum_i \left(f^{(i)} - y^{(i)} \right) x_j^{(i)}$$

Next: Module 2

Neural networks. Every idea from Module 1 — the model/cost/gradient recipe, feature scaling, the learning rate, overfitting, regularization — carries over unchanged. Only the model gets bigger.

1. Define underfitting and overfitting, and give the bias/variance name for each. State one symptom that distinguishes them. [6]
2. Seven data points are fitted by a degree-6 polynomial and $J_{\text{train}} = 0$. Explain why this happens and why it is not good news. [6]
3. Explain the term “high variance” by describing what happens if the training set is redrawn and the model refitted. [5]
4. Why can overfitting not be detected from the training cost? What would you need instead? [5]
5. List the three remedies for overfitting given in this lecture, and state one limitation of each. [6]
6. Write the regularized cost for linear regression. Explain the role of λ and why the penalty is divided by $2m$. [6]

7. Why is b excluded from the regularization term? [4]
8. Describe the fitted model when $\lambda = 0$ and when $\lambda = 10^{10}$, and name the failure mode in each case. [6]
9. Derive $\partial J / \partial w_j$ for the regularized squared-error cost, and state how the update rule changes. [6]
10. Show that the regularized update can be written as $w_j := (1 - \alpha\lambda/m)w_j - \alpha \partial J_{\text{fit}} / \partial w_j$, and explain the name “weight decay”. For $\alpha = 0.01$, $\lambda = 1$, $m = 50$, compute the shrink factor and its value after 1000 iterations. [8]
11. Write the regularized cost and gradient for logistic regression, and state exactly what differs from the linear case. [6]
12. Why must features be scaled before regularization is applied? Illustrate with two features measured in square feet and in number of rooms. [6]
13. Contrast L1 and L2 penalties. When would you prefer L1? [5]

1. **Design a λ search.** You have $m = 800$ examples and $n = 60$ features. Design a complete procedure for choosing λ : how you split the data, the grid you search, what you measure, and the stopping rule. State what you would report to justify the chosen value. [12]
2. **Diagnose from symptoms alone.** For each case, decide underfitting or overfitting and give two concrete next actions: (a) $J_{\text{train}} = 0.02$, $J_{\text{new}} = 0.9$; (b) $J_{\text{train}} = 0.55$, $J_{\text{new}} = 0.58$, both far above what a human achieves; (c) $J_{\text{train}} = 0.31$, $J_{\text{new}} = 0.33$, and a human achieves 0.30. [12]
3. **Design under a data constraint.** A rural clinic can supply only 150 labelled records but records 80 measurements per patient. Propose a modelling strategy, state which of the three remedies you rely on and why, and identify the risk your strategy does not remove. [12]
4. **Design a regularization unit test.** Write tests that would catch: b being penalised, the penalty being added to the cost but not the gradient, and λ being applied before feature scaling. For each, give the input and the expected observable difference. [12]
5. **Argue the trade-off to a stakeholder.** A hospital director asks why you deployed the model with 94% training accuracy rather than the one with 100%. Write the explanation you would give, in language that uses no equations. [10]

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

- **§2.2** — assessing model accuracy, the bias–variance trade-off, and why training error falls as flexibility rises.
- **§6.2.1–6.2.2** — ridge regression and the lasso: the penalty, the role of the tuning parameter, and why the predictors must be standardised.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

- **§1.1** and **§3.1.4** — polynomial curve fitting with a regularization term, and regularized least squares in matrix form.

Lab (CSE 816, Week 3): gradient descent for logistic regression; logistic regression with scikit-learn; overfitting; regularization.

Practice quizzes: *Gradient descent for logistic regression*; *The problem of overfitting*.